

Assignment 4: Text and Sequence Data

Curtis Meister

2026-04-03

Contents

```
library(keras)
library(tensorflow)
library(tfdatasets)
library(reticulate)
library(dplyr)
library(purrr)
library(stringr)
library(tibble)
library(tidyr)
library(ggplot2)
library(knitr)
library(rappdirs)
```

```
set.seed(123)
tf$random$set_seed(as.integer(123))
```

Per the instructions I have made the following adjustments:

```
max_words <- 10000
max_length <- 150
train_n <- 100
val_n <- 10000
batch_size <- 32
embed_dim <- 100
lstm_units <- 32
epochs_main <- 15
```

```
imdb <- dataset_imdb(num_words = max_words)

c(c(train_data, train_labels), c(test_data, test_labels)) %<-% imdb

cat("Original training sequences:", length(train_data), "\n")
```

```
## Original training sequences: 25000
```

```
cat("Original test sequences      :", length(test_data), "\n")
```

```
## Original test sequences      : 25000
```

```
word_index <- dataset_imdb_word_index()
reverse_word_index <- names(word_index)
names(reverse_word_index) <- word_index

decode_review <- function(review) {
  tokens <- sapply(review, function(i) {
    key <- as.character(i - 3)
    if (key %in% names(reverse_word_index)) {
      reverse_word_index[[key]]
    } else {
      "[UNK]"
    }
  })
  paste(tokens, collapse = " ")
}
```

```
train_text_150 <- map_chr(train_data, ~ decode_review(head(.x, max_length)))
test_text_150 <- map_chr(test_data, ~ decode_review(head(.x, max_length)))
```

The first 100 training samples are used for model fitting. The next 10,000 samples are for validation.

```
x_train_text <- train_text_150[1:train_n]
y_train <- train_labels[1:train_n]

x_val_text <- train_text_150[(train_n + 1):(train_n + val_n)]
y_val <- train_labels[(train_n + 1):(train_n + val_n)]

x_test_text <- test_text_150
y_test <- test_labels

cat("Training samples      :", length(x_train_text), "\n")
```

```
## Training samples      : 100
```

```
cat("Validation samples:", length(x_val_text), "\n")
```

```
## Validation samples: 10000
```

```
cat("Test samples      :", length(x_test_text), "\n")
```

```
## Test samples      : 25000
```

```
text_vectorization <- layer_text_vectorization(
  max_tokens = max_words,
  output_mode = "int",
  output_sequence_length = max_length
```

```

)

text_vectorization %>% adapt(x_train_text)

train_sequences <- text_vectorization(x_train_text)
val_sequences   <- text_vectorization(x_val_text)
test_sequences  <- text_vectorization(x_test_text)

vocab <- get_vocabulary(text_vectorization)
word_index_tv <- setNames(seq_along(vocab) - 1L, vocab)

cat("Vocabulary size from TextVectorization:", length(vocab), "\n")

## Vocabulary size from TextVectorization: 2634

train_ds <- tensor_slices_dataset(list(train_sequences, y_train)) %>%
  dataset_batch(batch_size)

val_ds <- tensor_slices_dataset(list(val_sequences, y_val)) %>%
  dataset_batch(batch_size)

test_ds <- tensor_slices_dataset(list(test_sequences, y_test)) %>%
  dataset_batch(batch_size)

build_learned_model <- function(max_words = 10000,
                                embed_dim = 100,
                                lstm_units = 32,
                                dropout_rate = 0.5) {

  inputs <- layer_input(shape = c(max_length), dtype = "int64")

  outputs <- inputs %>%
    layer_embedding(
      input_dim = max_words,
      output_dim = embed_dim,
      mask_zero = TRUE
    ) %>%
    bidirectional(layer_lstm(units = lstm_units)) %>%
    layer_dropout(rate = dropout_rate) %>%
    layer_dense(units = 1, activation = "sigmoid")

  keras_model(inputs = inputs, outputs = outputs) %>%
    compile(
      optimizer = "rmsprop",
      loss = "binary_crossentropy",
      metrics = "accuracy"
    )
}

learned_model <- build_learned_model(
  max_words = max_words,
  embed_dim = embed_dim,

```

```

    lstm_units = lstm_units
)

summary(learned_model)

```

```

## Model: "model"
## -----
## Layer (type)                Output Shape          Param #
## =====
## input_1 (InputLayer)        [(None, 150)]         0
## embedding (Embedding)       (None, 150, 100)      1000000
## bidirectional (Bidirectional) (None, 64)            34048
## dropout (Dropout)           (None, 64)            0
## dense (Dense)                (None, 1)             65
## =====
## Total params: 1034113 (3.94 MB)
## Trainable params: 1034113 (3.94 MB)
## Non-trainable params: 0 (0.00 Byte)
## -----

```

```

history_learned <- learned_model %>% fit(
  train_ds,
  validation_data = val_ds,
  epochs = epochs_main,
  callbacks = list(
    callback_early_stopping(
      monitor = "val_accuracy",
      patience = 3,
      restore_best_weights = TRUE
    )
  ),
  verbose = 2
)

```

```

## Epoch 1/15
## 4/4 - 27s - loss: 0.6911 - accuracy: 0.5800 - val_loss: 0.6938 - val_accuracy: 0.4948 - 27s/epoch - 1s
## Epoch 2/15
## 4/4 - 5s - loss: 0.6824 - accuracy: 0.5800 - val_loss: 0.6947 - val_accuracy: 0.4948 - 5s/epoch - 1s
## Epoch 3/15
## 4/4 - 5s - loss: 0.6737 - accuracy: 0.5800 - val_loss: 0.6960 - val_accuracy: 0.4948 - 5s/epoch - 1s
## Epoch 4/15
## 4/4 - 5s - loss: 0.6726 - accuracy: 0.5800 - val_loss: 0.6964 - val_accuracy: 0.4948 - 5s/epoch - 1s

```

```

learned_eval <- learned_model %>% evaluate(test_ds, verbose = 0)
learned_eval

```

```

##      loss  accuracy
## 0.6935223 0.5000000

```

```

options(timeout = max(1200, getOption("timeout")))

```

```

glove_dir <- file.path(path.expand("~"), "glove")
dir.create(glove_dir, recursive = TRUE, showWarnings = FALSE)

zip_path <- file.path(glove_dir, "glove.6B.zip")
glove_txt <- file.path(glove_dir, "glove.6B.100d.txt")

if (!file.exists(glove_txt)) {
  zip_ok <- FALSE

  if (file.exists(zip_path)) {
    zip_ok <- tryCatch({
      tmp <- utils::unzip(zip_path, list = TRUE)
      any(grepl("glove.6B.100d.txt", tmp$Name))
    }, error = function(e) FALSE)
  }

  if (!zip_ok) {
    if (file.exists(zip_path)) {
      cat("Existing zip is invalid. Removing partial/corrupt file...\n")
      file.remove(zip_path)
    }

    cat("Downloading GloVe zip to:", zip_path, "\n")
    download.file(
      url = "https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip",
      destfile = zip_path,
      mode = "wb",
      method = "libcurl"
    )
  }

  cat("Extracting glove.6B.100d.txt from zip...\n")
  utils::unzip(zip_path, files = "glove.6B.100d.txt", exdir = glove_dir)
}

cat("glove_dir exists :", dir.exists(glove_dir), "\n")

```

```
## glove_dir exists : TRUE
```

```
cat("zip exists      :", file.exists(zip_path), "\n")
```

```
## zip exists      : TRUE
```

```
cat("txt exists      :", file.exists(glove_txt), "\n")
```

```
## txt exists      : TRUE
```

```
cat("Files in glove_dir:\n")
```

```
## Files in glove_dir:
```

```
print(list.files(glove_dir))
```

```
## [1] "glove.6B.100d.txt" "glove.6B.zip"
```

```
use_pretrained_glove <- file.exists(glove_txt)
```

```
if (use_pretrained_glove) {  
  cat("Using GloVe file:", glove_txt, "\n")  
} else {  
  cat("GloVe file was not found - continuing with fallback random matrix.\n")  
}
```

```
## Using GloVe file: /Users/curtismeister/glove/glove.6B.100d.txt
```

```
embeddings_index <- new.env(hash = TRUE, parent = emptyenv())
```

```
if (use_pretrained_glove) {  
  con <- file(glove_txt, open = "r")  
  while (length(line <- readLines(con, n = 1, warn = FALSE)) > 0) {  
    values <- strsplit(line, " ")[[1]]  
    word <- values[1]  
    coefs <- as.numeric(values[-1])  
    embeddings_index[[word]] <- coefs  
  }  
  close(con)  
  cat("Loaded", length(ls(embeddings_index)), "GloVe word vectors\n")  
} else {  
  cat("Loaded 0 GloVe word vectors\n")  
}
```

```
## Loaded 399187 GloVe word vectors
```

```
if (use_pretrained_glove) {  
  embedding_matrix <- matrix(0, nrow = max_words, ncol = embed_dim)  
  
  # fill the matrix with GloVe vectors when the word is found  
  for (word in vocab) {  
    idx <- unname(word_index_tv[word])  
  
    if (length(idx) == 1 && !is.na(idx) && idx > 0 && idx < max_words) {  
      vec <- embeddings_index[[word]]  
  
      if (!is.null(vec) && length(vec) == embed_dim) {  
        embedding_matrix[idx + 1, ] <- vec  
      }  
    }  
  }  
  
  coverage <- sum(rowSums(abs(embedding_matrix)) > 0)  
  cat("Words covered by GloVe:", coverage, "\n")  
} else {
```

```

set.seed(123)
embedding_matrix <- matrix(
  rnorm(max_words * embed_dim, mean = 0, sd = 0.02),
  nrow = max_words,
  ncol = embed_dim
)
embedding_matrix[1, ] <- 0
coverage <- 0
cat("Using fallback random embedding matrix because GloVe was unavailable.\n")
}

```

Words covered by GloVe: 2625

```

build_pretrained_model <- function(embedding_matrix,
                                   max_words = 10000,
                                   embed_dim = 100,
                                   lstm_units = 32,
                                   dropout_rate = 0.5) {

  inputs <- layer_input(shape = c(max_length), dtype = "int64")

  outputs <- inputs %>%
    layer_embedding(
      input_dim = max_words,
      output_dim = embed_dim,
      weights = list(embedding_matrix),
      trainable = FALSE,
      mask_zero = TRUE
    ) %>%
    bidirectional(layer_lstm(units = lstm_units)) %>%
    layer_dropout(rate = dropout_rate) %>%
    layer_dense(units = 1, activation = "sigmoid")

  keras_model(inputs = inputs, outputs = outputs) %>%
    compile(
      optimizer = "rmsprop",
      loss = "binary_crossentropy",
      metrics = "accuracy"
    )
}

```

```

pretrained_model <- build_pretrained_model(
  embedding_matrix = embedding_matrix,
  max_words = max_words,
  embed_dim = embed_dim,
  lstm_units = lstm_units
)

summary(pretrained_model)

```

Model: "model_1"

```

## Layer (type)                Output Shape                Param #   Trainable
## =====
## input_2 (InputLayer)        [(None, 150)]               0         Y
## embedding_1 (Embedding)     (None, 150, 100)           1000000   N
## bidirectional_1 (Bidirection (None, 64)               34048     Y
## al)
## dropout_1 (Dropout)         (None, 64)                  0         Y
## dense_1 (Dense)             (None, 1)                   65        Y
## =====
## Total params: 1034113 (3.94 MB)
## Trainable params: 34113 (133.25 KB)
## Non-trainable params: 1000000 (3.81 MB)
## -----

```

```

history_pretrained <- pretrained_model %>% fit(
  train_ds,
  validation_data = val_ds,
  epochs = epochs_main,
  callbacks = list(
    callback_early_stopping(
      monitor = "val_accuracy",
      patience = 3,
      restore_best_weights = TRUE
    )
  ),
  verbose = 2
)

```

```

## Epoch 1/15
## 4/4 - 14s - loss: 0.7005 - accuracy: 0.5500 - val_loss: 0.6935 - val_accuracy: 0.4999 - 14s/epoch - 4
## Epoch 2/15
## 4/4 - 5s - loss: 0.6502 - accuracy: 0.6600 - val_loss: 0.6941 - val_accuracy: 0.5050 - 5s/epoch - 1s
## Epoch 3/15
## 4/4 - 5s - loss: 0.6396 - accuracy: 0.6400 - val_loss: 0.6912 - val_accuracy: 0.5162 - 5s/epoch - 1s
## Epoch 4/15
## 4/4 - 5s - loss: 0.6620 - accuracy: 0.6500 - val_loss: 0.6934 - val_accuracy: 0.5136 - 5s/epoch - 1s
## Epoch 5/15
## 4/4 - 5s - loss: 0.6514 - accuracy: 0.6800 - val_loss: 0.6915 - val_accuracy: 0.5255 - 5s/epoch - 1s
## Epoch 6/15
## 4/4 - 5s - loss: 0.6131 - accuracy: 0.6800 - val_loss: 0.6895 - val_accuracy: 0.5347 - 5s/epoch - 1s
## Epoch 7/15
## 4/4 - 5s - loss: 0.6501 - accuracy: 0.6000 - val_loss: 0.6899 - val_accuracy: 0.5396 - 5s/epoch - 1s
## Epoch 8/15
## 4/4 - 5s - loss: 0.6103 - accuracy: 0.6600 - val_loss: 0.6917 - val_accuracy: 0.5350 - 5s/epoch - 1s
## Epoch 9/15
## 4/4 - 5s - loss: 0.5762 - accuracy: 0.7500 - val_loss: 0.6918 - val_accuracy: 0.5374 - 5s/epoch - 1s
## Epoch 10/15
## 4/4 - 5s - loss: 0.5915 - accuracy: 0.7600 - val_loss: 0.6921 - val_accuracy: 0.5425 - 5s/epoch - 1s
## Epoch 11/15
## 4/4 - 5s - loss: 0.5829 - accuracy: 0.7000 - val_loss: 0.6933 - val_accuracy: 0.5418 - 5s/epoch - 1s
## Epoch 12/15
## 4/4 - 6s - loss: 0.5522 - accuracy: 0.7500 - val_loss: 0.6967 - val_accuracy: 0.5432 - 6s/epoch - 1s
## Epoch 13/15
## 4/4 - 6s - loss: 0.5409 - accuracy: 0.7800 - val_loss: 0.7002 - val_accuracy: 0.5438 - 6s/epoch - 2s

```



```
## Epoch 14/15
## 4/4 - 6s - loss: 0.5328 - accuracy: 0.7600 - val_loss: 0.6980 - val_accuracy: 0.5493 - 6s/epoch - 2s
## Epoch 15/15
## 4/4 - 6s - loss: 0.4741 - accuracy: 0.8200 - val_loss: 0.7001 - val_accuracy: 0.5521 - 6s/epoch - 2s
```

```
pretrained_eval <- pretrained_model %>% evaluate(test_ds, verbose = 0)
pretrained_eval
```

```
##      loss accuracy
## 0.7011006 0.5444400
```

Results at 100 Training Samples

```
comparison <- tibble(
  Model = c("Learned Embedding", "Pretrained GloVe"),
  Test_Accuracy = c(as.numeric(learned_eval[["accuracy"]]),
                    as.numeric(pretrained_eval[["accuracy"]])),
  Test_Loss = c(as.numeric(learned_eval[["loss"]]),
                as.numeric(pretrained_eval[["loss"]]))
)

knitr::kable(
  comparison,
  col.names = c("Model", "Test Accuracy", "Test Loss"),
  digits = 4,
  caption = "Results (100 Samples)"
)
```

Table 1: Results (100 Samples)

Model	Test Accuracy	Test Loss
Learned Embedding	0.5000	0.6935
Pretrained GloVe	0.5444	0.7011

```
training_sizes <- c(100, 200, 500, 1000, 2000)
results <- tibble()
```

```
for (n_train in training_sizes) {

  x_tr_n <- train_text_150[1:n_train]
  y_tr_n <- train_labels[1:n_train]

  tv_n <- layer_text_vectorization(
    max_tokens = max_words,
    output_mode = "int",
    output_sequence_length = max_length
  )

  tv_n %>% adapt(x_tr_n)

  tr_seq_n <- tv_n(x_tr_n)
```

```

val_seq_n <- tv_n(x_val_text)
te_seq_n <- tv_n(x_test_text)

vocab_n <- get_vocabulary(tv_n)
word_idx_n <- setNames(seq_along(vocab_n) - 1L, vocab_n)

tr_ds_n <- tensor_slices_dataset(list(tr_seq_n, y_tr_n)) %>%
  dataset_batch(batch_size)
val_ds_n <- tensor_slices_dataset(list(val_seq_n, y_val)) %>%
  dataset_batch(batch_size)
te_ds_n <- tensor_slices_dataset(list(te_seq_n, y_test)) %>%
  dataset_batch(batch_size)

early_stop <- list(
  callback_early_stopping(
    monitor = "val_accuracy",
    patience = 2,
    restore_best_weights = TRUE
  )
)

model_learned_n <- build_learned_model(
  max_words = max_words,
  embed_dim = embed_dim,
  lstm_units = lstm_units
)

model_learned_n %>% fit(
  tr_ds_n,
  validation_data = val_ds_n,
  epochs = 10,
  callbacks = early_stop,
  verbose = 0
)

learned_metrics_n <- model_learned_n %>% evaluate(te_ds_n, verbose = 0)

if (use_pretrained_glove) {
  embedding_matrix_n <- matrix(0, nrow = max_words, ncol = embed_dim)

  for (word in vocab_n) {
    idx <- unname(word_idx_n[word])

    if (length(idx) == 1 && !is.na(idx) && idx > 0 && idx < max_words) {
      vec <- embeddings_index[[word]]
      if (!is.null(vec) && length(vec) == embed_dim) {
        embedding_matrix_n[idx + 1, ] <- vec
      }
    }
  }
} else {
  set.seed(123)
  embedding_matrix_n <- matrix(

```

```

    rnorm(max_words * embed_dim, mean = 0, sd = 0.02),
    nrow = max_words,
    ncol = embed_dim
  )
  embedding_matrix_n[1, ] <- 0
}

model_pretrained_n <- build_pretrained_model(
  embedding_matrix = embedding_matrix_n,
  max_words = max_words,
  embed_dim = embed_dim,
  lstm_units = lstm_units
)

model_pretrained_n %>% fit(
  tr_ds_n,
  validation_data = val_ds_n,
  epochs = 10,
  callbacks = early_stop,
  verbose = 0
)

pretrained_metrics_n <- model_pretrained_n %>% evaluate(te_ds_n, verbose = 0)

results <- bind_rows(
  results,
  tibble(
    training_size = n_train,
    model = "Learned Embedding",
    test_loss = as.numeric(learned_metrics_n[["loss"]]),
    test_accuracy = as.numeric(learned_metrics_n[["accuracy"]])
  ),
  tibble(
    training_size = n_train,
    model = "Pretrained GloVe",
    test_loss = as.numeric(pretrained_metrics_n[["loss"]]),
    test_accuracy = as.numeric(pretrained_metrics_n[["accuracy"]])
  )
)
}

```

```

results_wide <- results %>%
  select(training_size, model, test_accuracy) %>%
  pivot_wider(names_from = model, values_from = test_accuracy)

knitr::kable(
  results_wide,
  col.names = c("Training Samples", "Learned Embedding", "Pretrained GloVe"),
  digits = 4,
  caption = "Accuracy vs. Training Size"
)

```

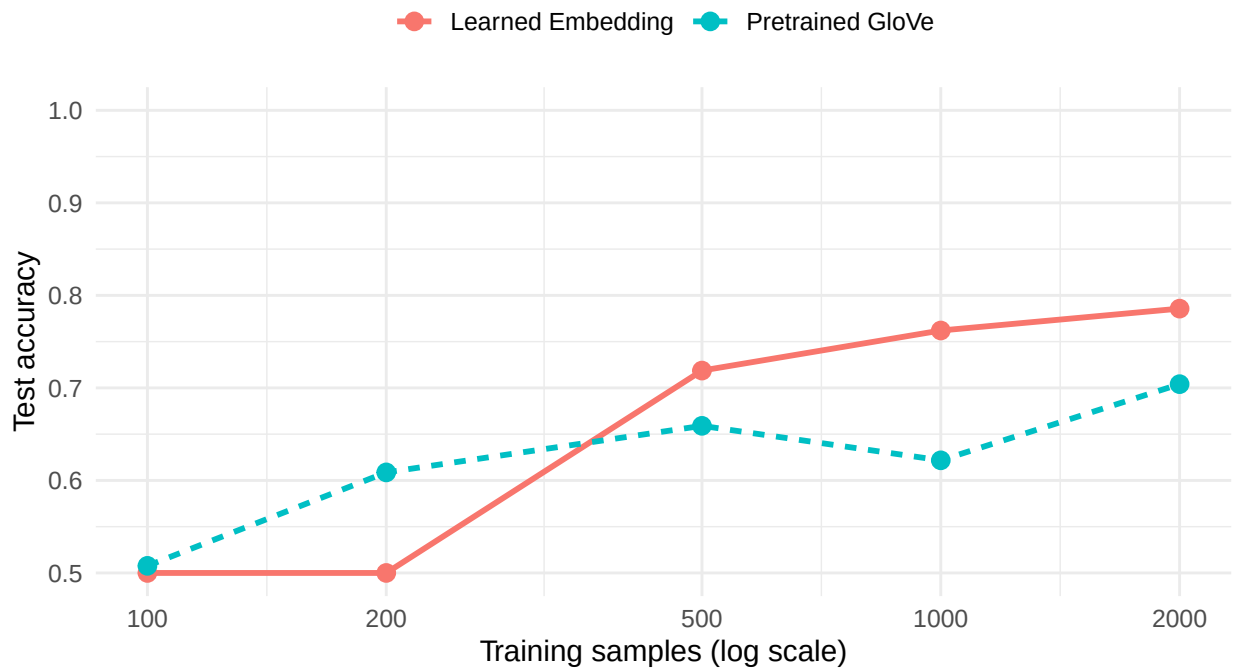
Table 2: Accuracy vs. Training Size

Training Samples	Learned Embedding	Pretrained GloVe
100	0.5000	0.5077
200	0.5000	0.6087
500	0.7187	0.6590
1000	0.7620	0.6217
2000	0.7857	0.7040

```
ggplot(results, aes(x = training_size, y = test_accuracy, color = model,
                    linetype = model, group = model)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 3) +
  scale_x_log10(breaks = training_sizes, labels = training_sizes) +
  scale_y_continuous(limits = c(0.5, 1.0)) +
  labs(
    title = "Accuracy vs. Training Sample Size",
    subtitle = "Learned vs. Pretrained GloVe",
    x = "Training samples (log scale)",
    y = "Test accuracy",
    color = NULL,
    linetype = NULL
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "top")
```

Accuracy vs. Training Sample Size

Learned vs. Pretrained GloVe



```

crossover_row <- results %>%
  select(training_size, model, test_accuracy) %>%
  pivot_wider(names_from = model, values_from = test_accuracy) %>%
  filter(`Learned Embedding` >= `Pretrained GloVe`) %>%
  arrange(training_size) %>%
  slice(1)

crossover_n <- if (nrow(crossover_row) > 0) {
  as.character(crossover_row$training_size)
} else {
  paste0("> ", max(training_sizes))
}

crossover_n

```

```
## [1] "500"
```